Web Security Testing Report

E-commerce REST API (Node.js, Express, Neon PostgreSQL)

BATMAN TECHNOLOGY - BATTECHNO: Web Security Fundamentals, Task 3

Student: Ahmad Tahat | University ID: 20220336

All tests were done on my own local project. No real system was tested.

1. Summary

The first version of the API worked, but it had no login, no roles and no protection. Anyone could read all users, create an admin and change products. I reviewed the project, found 14 problems, fixed them, and tested each fix in Postman.

The one good point in the first version was that all SQL queries already used parameters ($1, $2), so SQL Injection was not possible.

Critical	High	Medium	Low
2	4	4	4

Status: all 14 problems are fixed and tested.

Severity scale

Critical: full takeover or data loss is easy. High: serious damage is likely. Medium: damage is possible with extra steps. Low: small risk or bad practice.

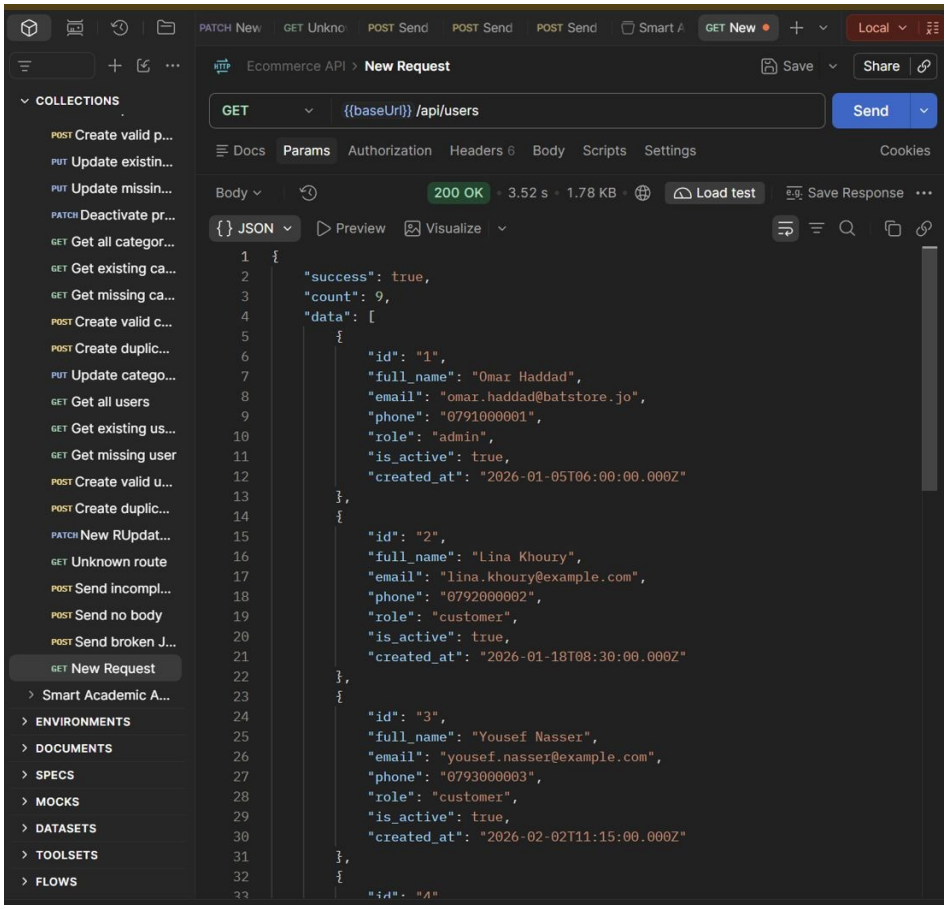
2. How I Tested

I ran the API locally and tested with a Postman collection (Web_Security_Testing.postman_collection.json). Each request has automatic tests for the expected status code. I tested both success cases and failure cases. Screenshots are in the Auth_Security_Evidence folder, with a list in INDEX.csv.

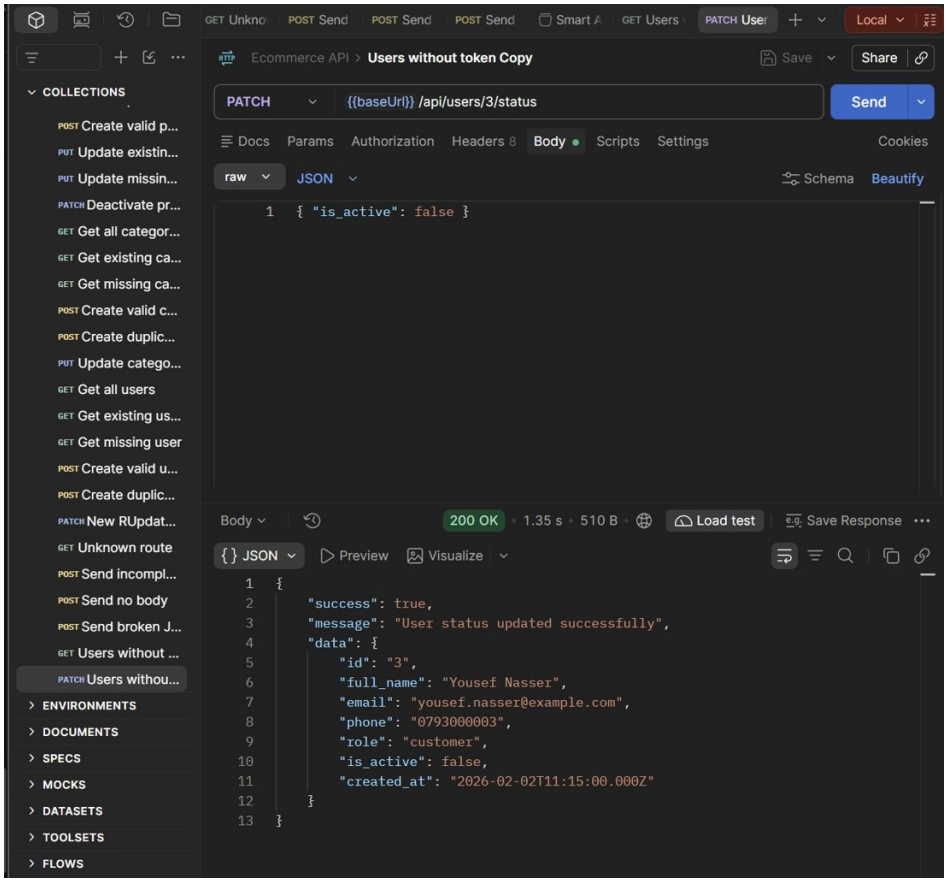
3. Findings (Before and After)

Finding 1: No Authentication

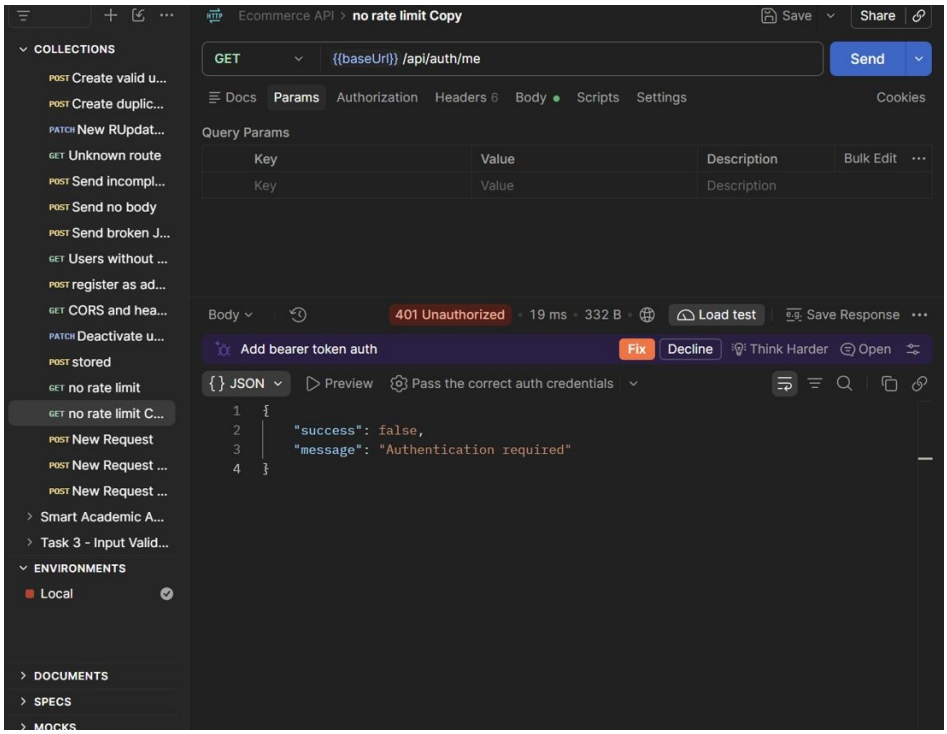
Severity	Critical. Anyone could read and change data without logging in.
Location	routes/usersRoutes.js, productsRoutes.js, categoriesRoutes.js
How I tested	Postman: GET /api/users and PATCH /api/users/:id/status with no token. Also code review.
Result before the fix	The server returned the full user list and let me deactivate a user with no login. Any stranger could do this.
How it was fixed	Added JWT login and the authenticate middleware (middleware/authenticate.js) on all protected routes.
Result after the fix	Requests with no token now return 401 Unauthorized.



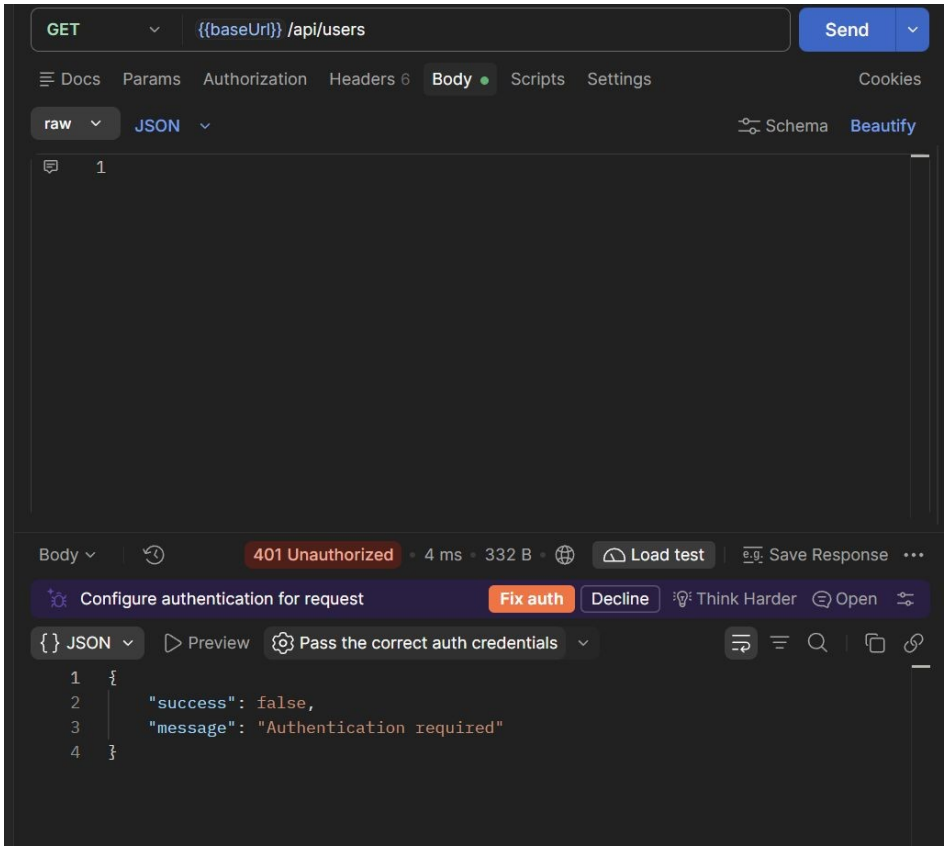
BEFORE: user list open without a token (01-users-no-token.png)



BEFORE: user deactivated without login (02-deactivate-user.png)



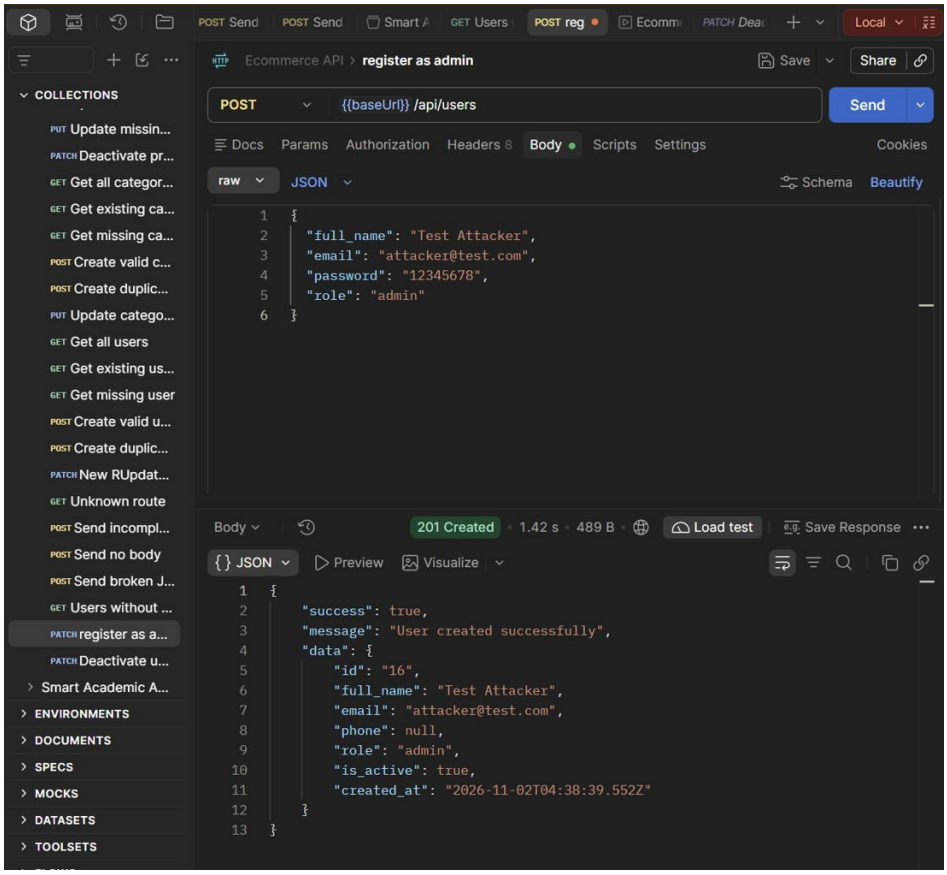
AFTER: /me without a token returns 401 (26-me-no-token-401.png)



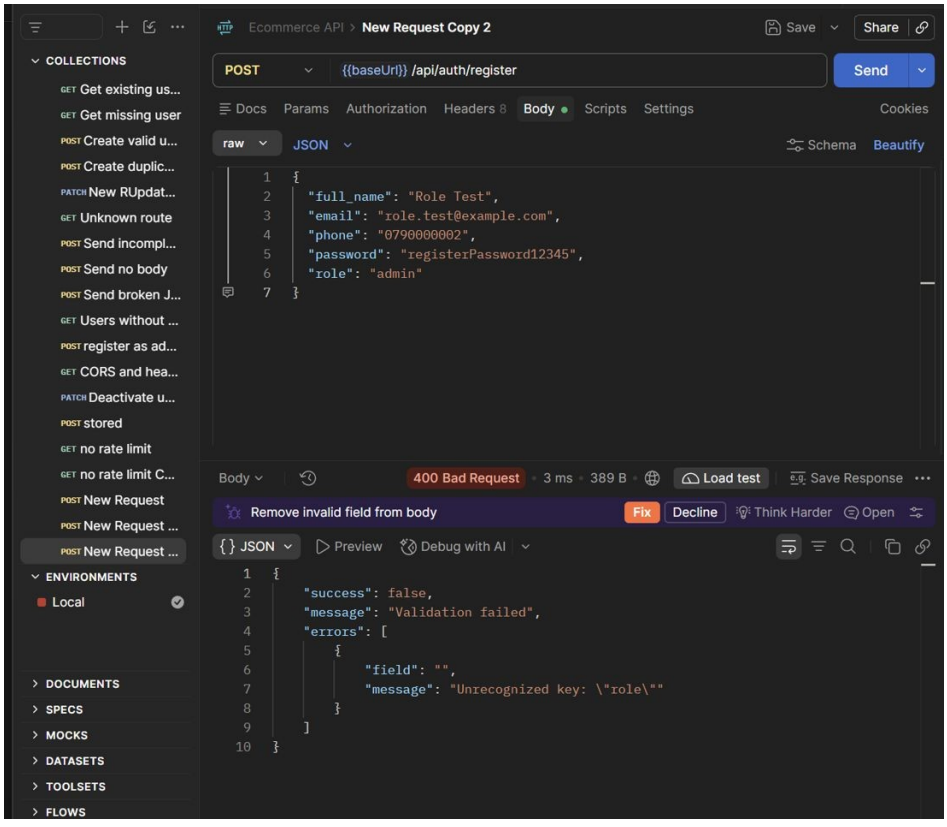
AFTER: user list without a token returns 401 (31-users-no-token-401.png)

Finding 2: Role Accepted From the Request Body (Privilege Escalation)

Severity	Critical. Anyone could make themselves an admin.
Location	usersController.createUser, POST /api/users
How I tested	Postman: POST a new user with "role":"admin".
Result before the fix	An admin account was created with no login. The attacker gets full control.
How it was fixed	Removed the public POST /api/users route. POST /api/auth/register has no role field, rejects unknown fields, and always saves the role "customer".
Result after the fix	The same request now returns 400 Validation failed.



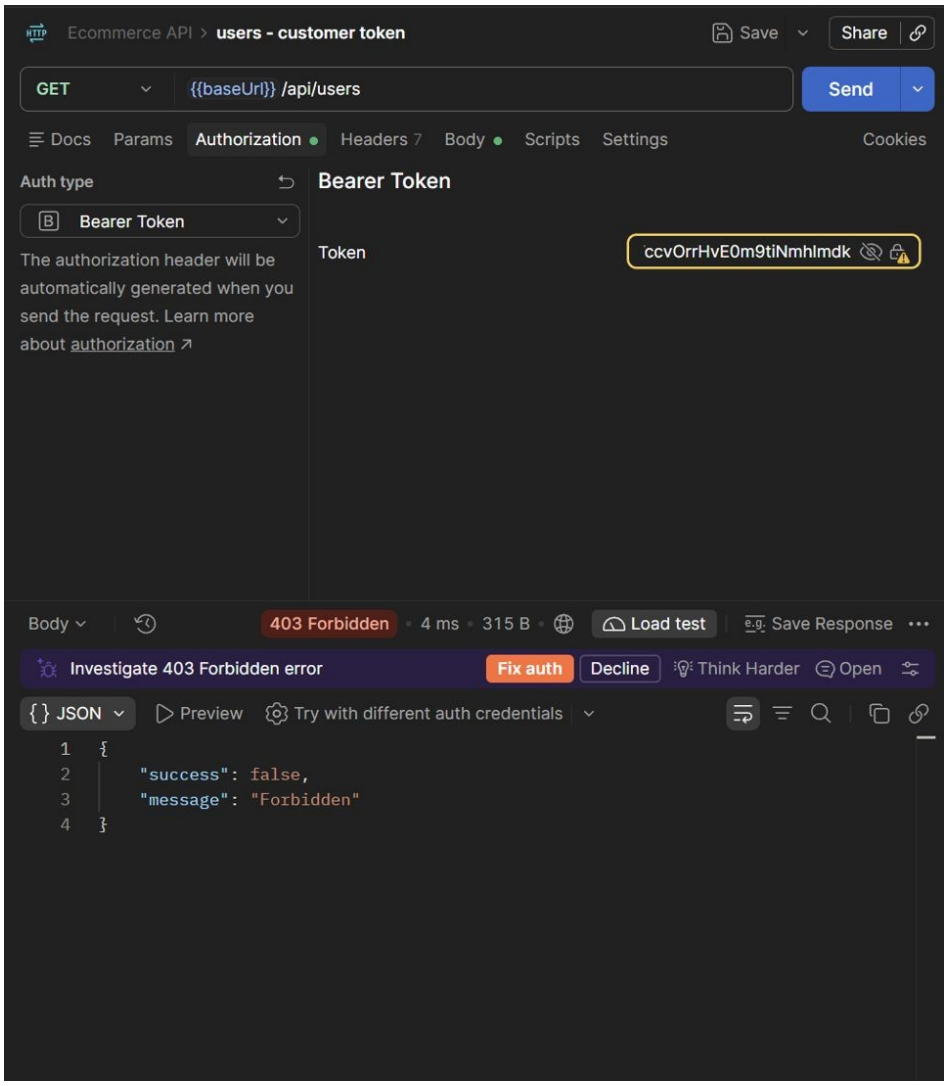
BEFORE: account created with role admin (03-register-as-admin.png)



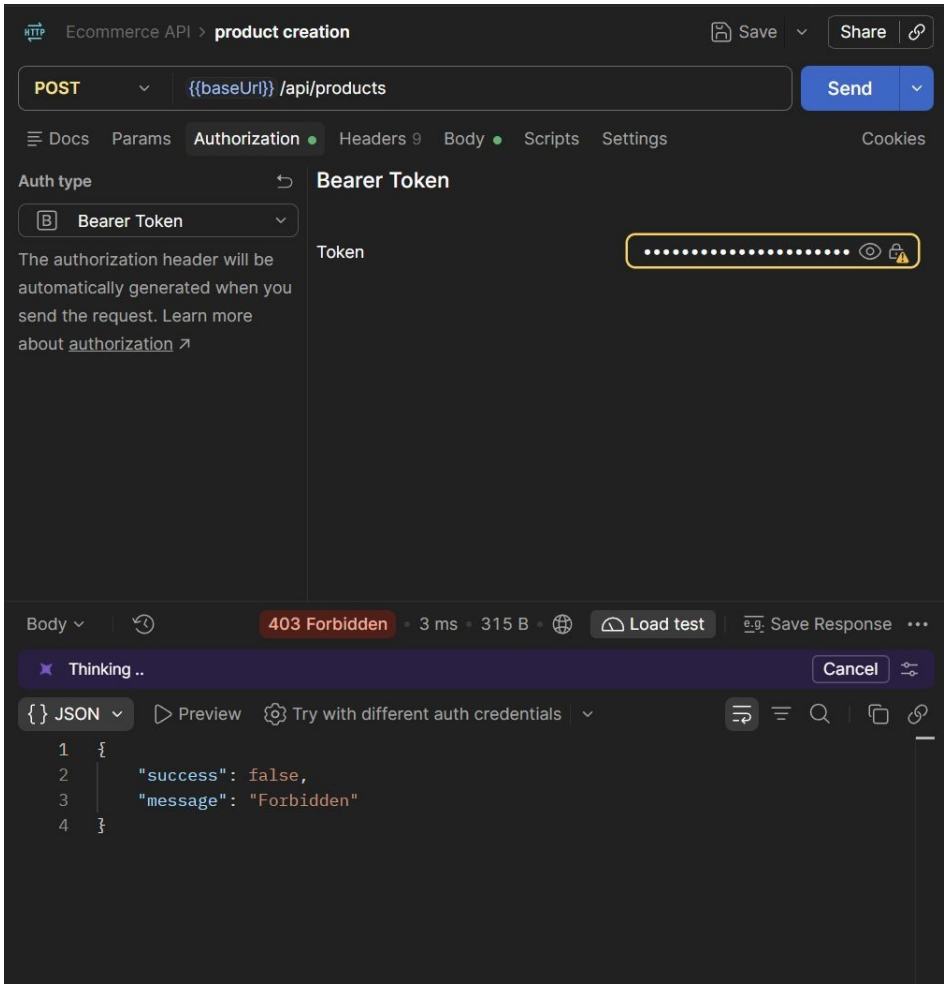
AFTER: role field rejected with 400 (22-register-role-rejected-400.png)

Finding 3: Missing Authorization (No Role Checks)

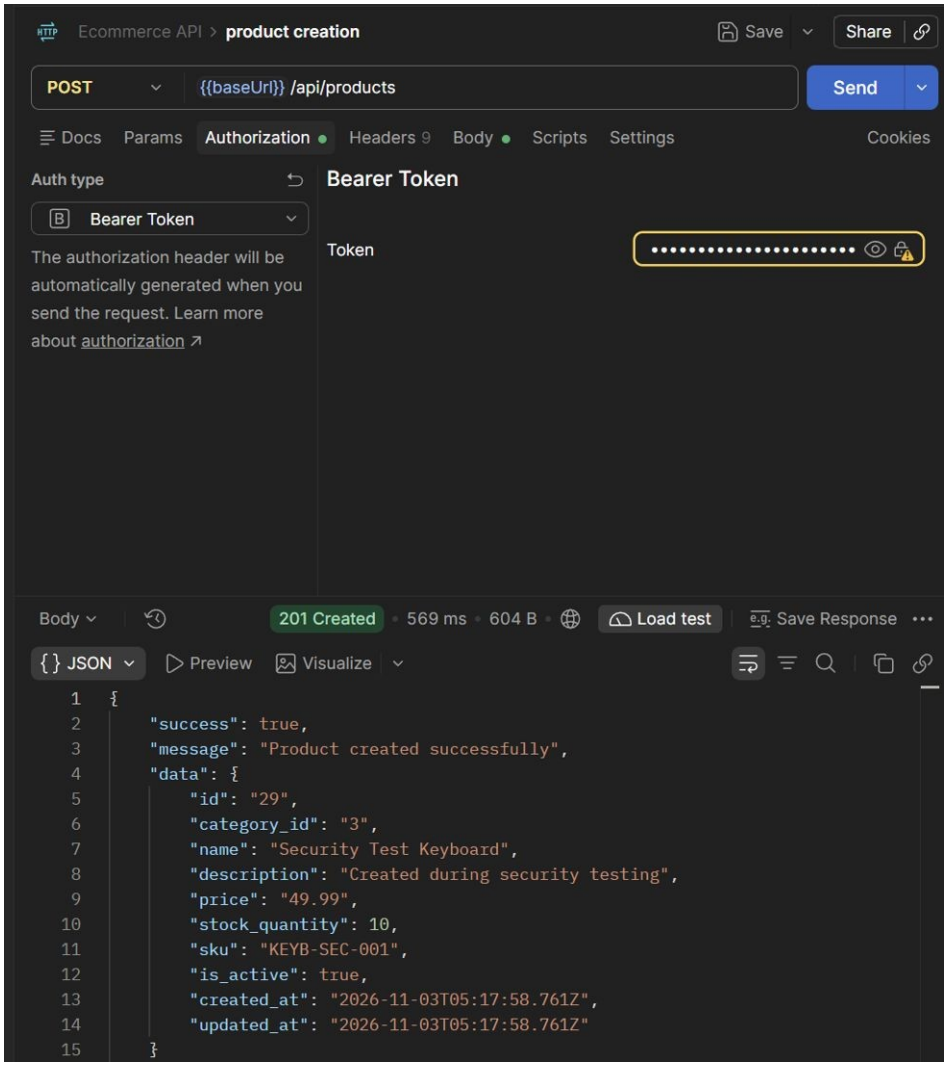
Severity	High. A normal user could do admin actions.
Location	Write routes for users, products and categories
How I tested	Postman: log in as a customer, then call admin routes.
Result before the fix	There was no role check, so any user could add or disable products.
How it was fixed	Added the authorize("admin") middleware (middleware/authorize.js).
Result after the fix	A customer gets 403 Forbidden. An admin gets 200 or 201.



AFTER: customer blocked from user list (403) (32-users-customer-403.png)



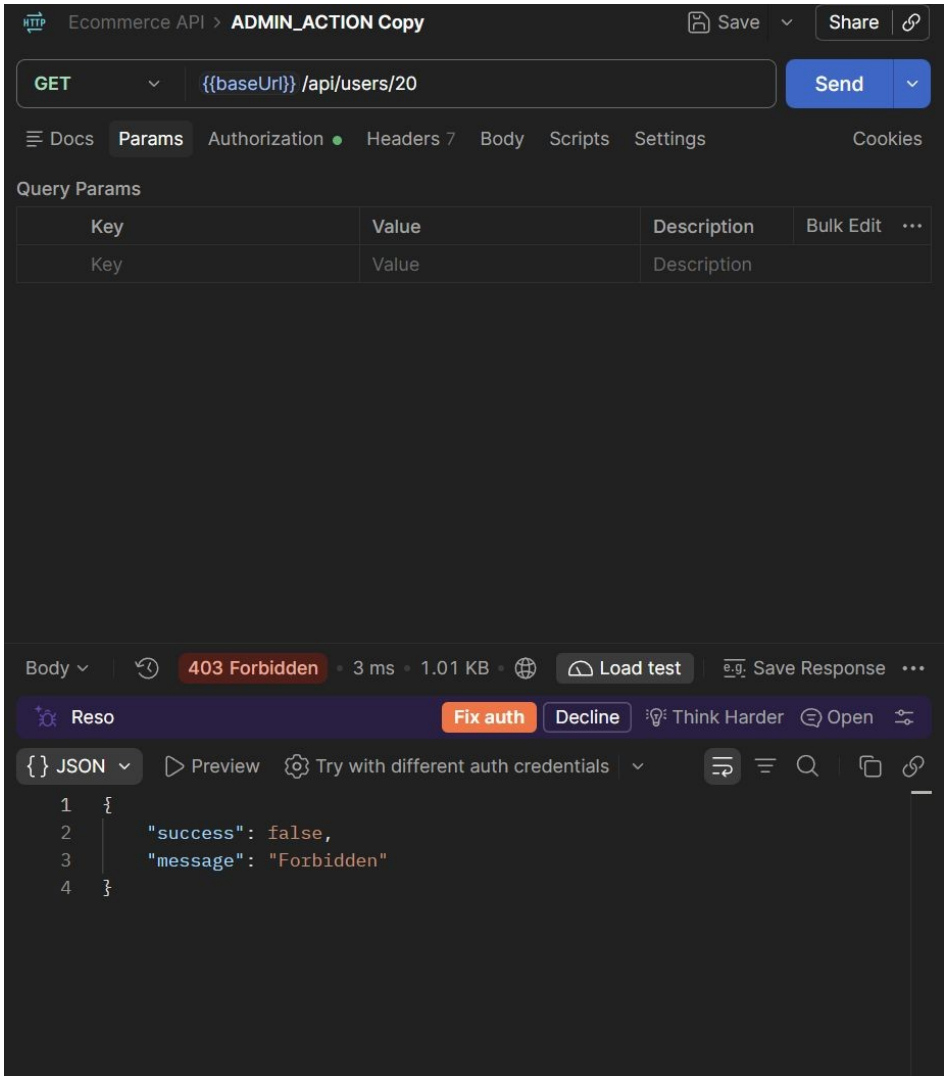
AFTER: customer cannot add a product (403) (36-product-create-customer-403.png)



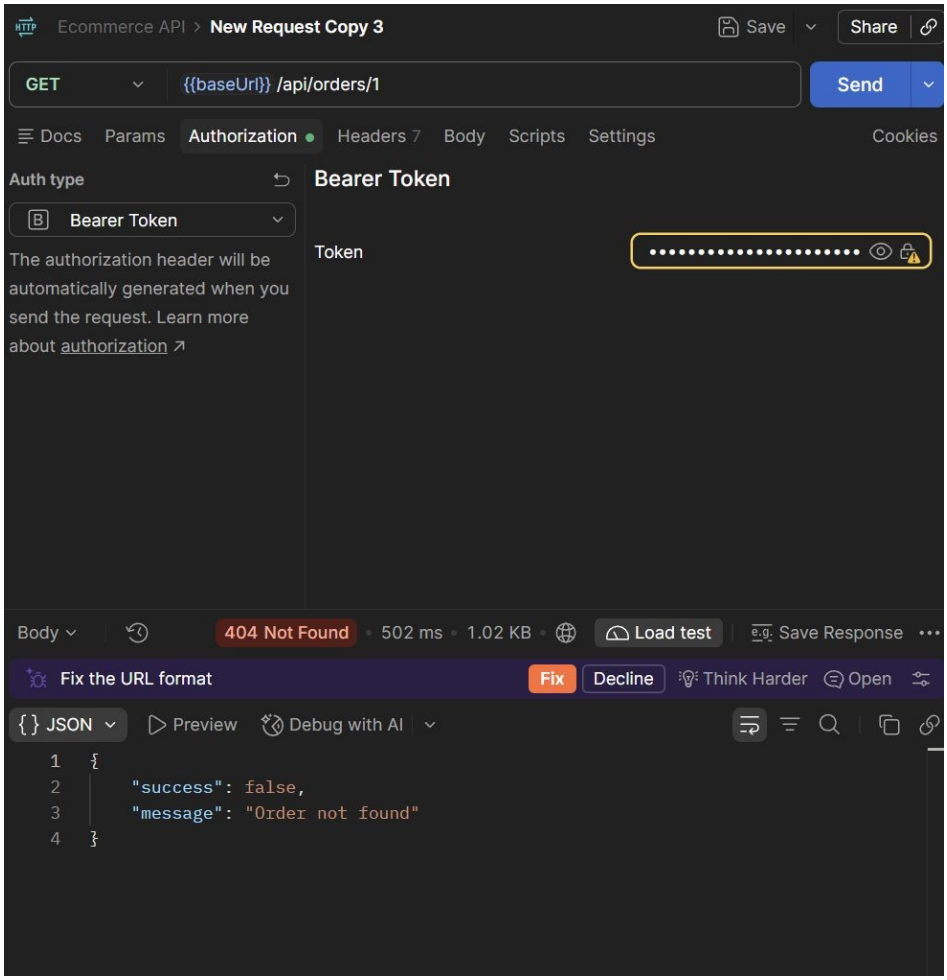
AFTER: admin adds a product (201) (35-product-create-admin-201.png)

Finding 4: IDOR (Access to Another User's Data)

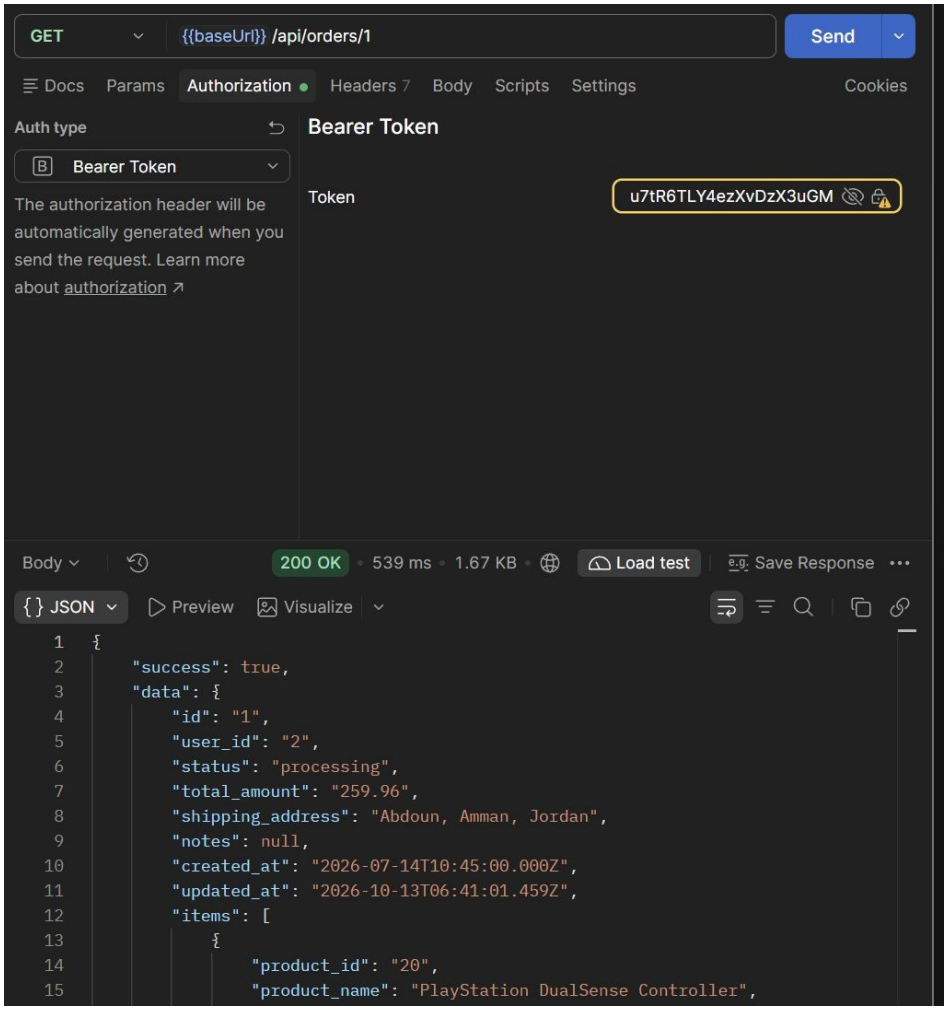
Severity	High. A user could read other people's private data and orders.
Location	GET /api/users/:id, GET /api/orders/:id
How I tested	Postman: log in as a customer and request another user's record and another user's order.
Result before the fix	The old API had no ownership check, so changing the ID in the URL showed someone else's data.
How it was fixed	Users can only read their own record (admin can read all). Orders are filtered by user_id inside the SQL query.
Result after the fix	Another user's record returns 403. Another user's order returns 404, so the API does not even confirm the order exists.



AFTER: another user's record (403) (46-user-other-record-403.png)



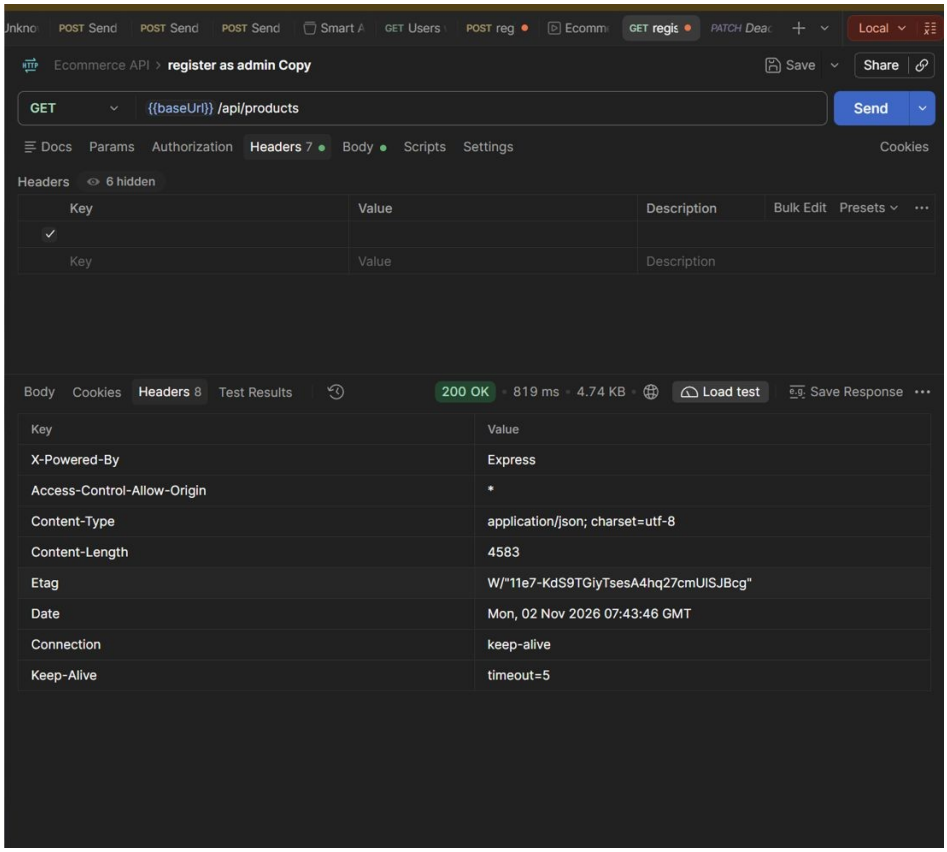
AFTER: another user's order (404) (50-order-other-user-404.png)



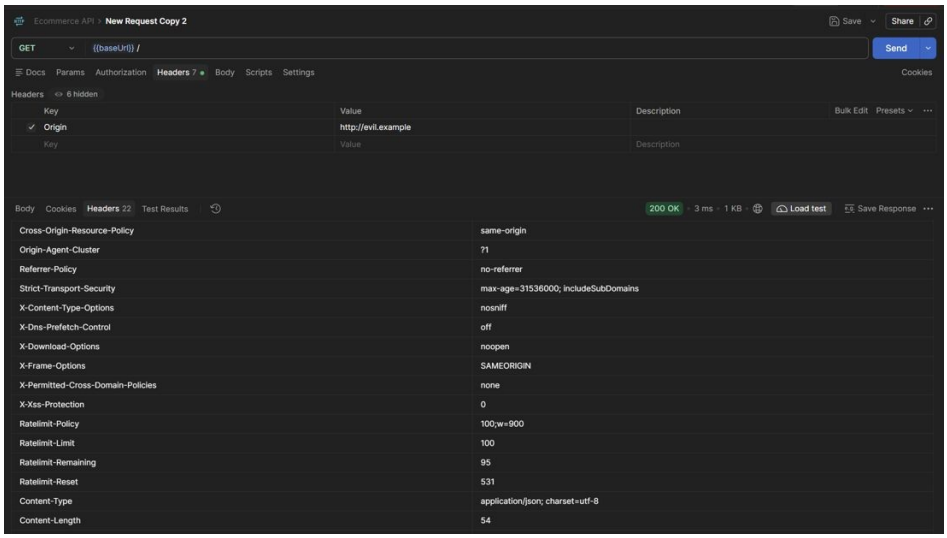
AFTER: admin can read the order (200) (51-order-admin-200.png)

Finding 5: Open CORS

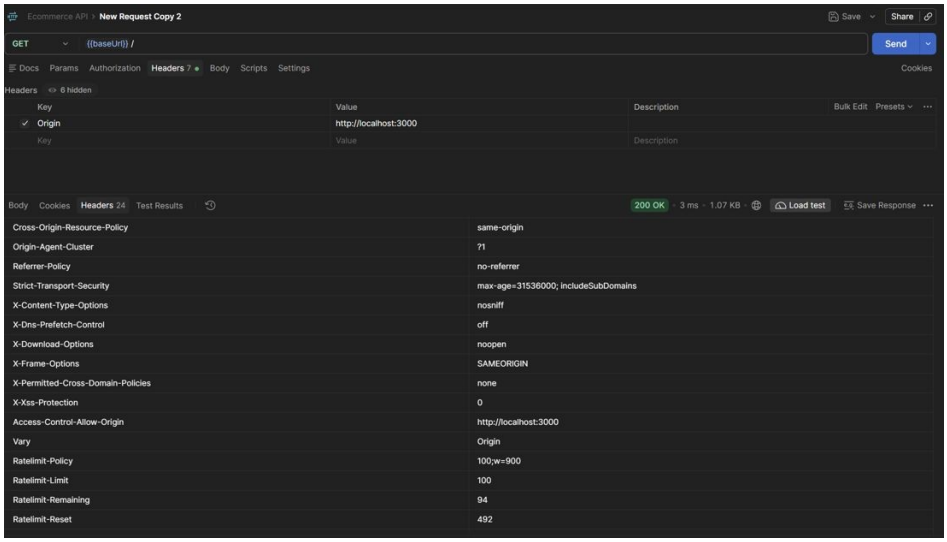
Severity	Medium. Any website could call the API from a browser.
Location	src/app.js
How I tested	Postman: send a request with an unknown Origin header and check the response headers.
Result before the fix	The API allowed every origin.
How it was fixed	CORS now uses an allow-list from ALLOWED_ORIGINS, with fixed methods and headers.
Result after the fix	An unknown origin gets no Access-Control-Allow-Origin header. An allowed origin gets it.



BEFORE: CORS allows every origin (04-cors-wildcard.png)



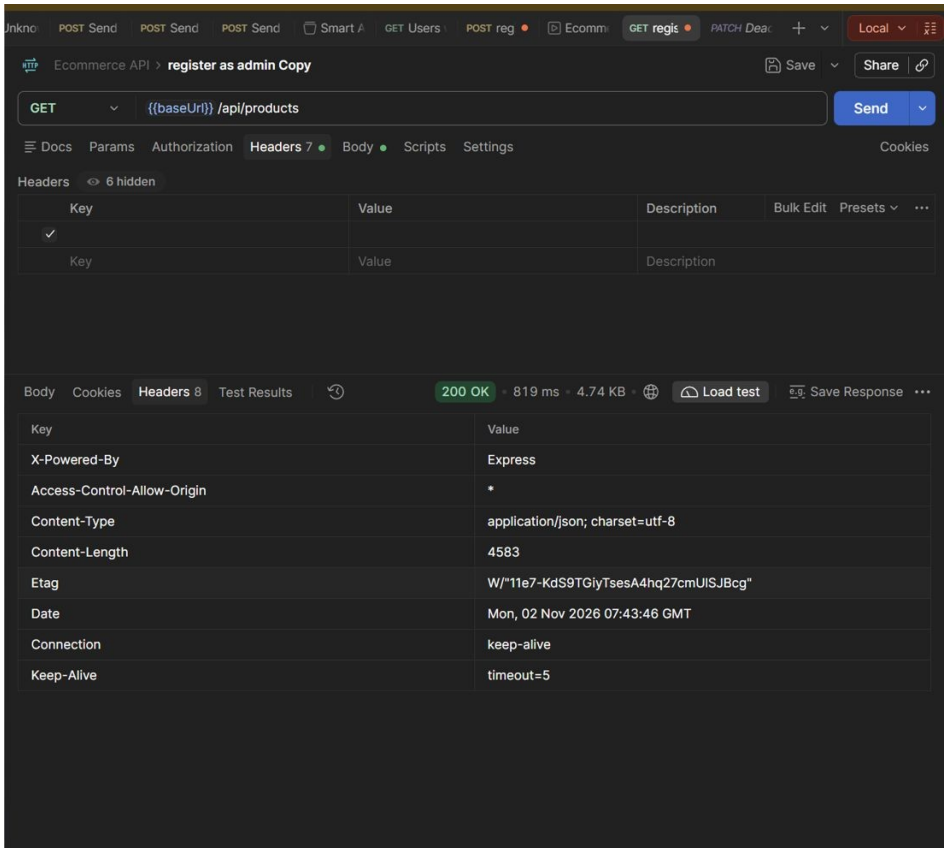
AFTER: unknown origin gets no CORS header (42-cors-evil-origin-no-header.png)



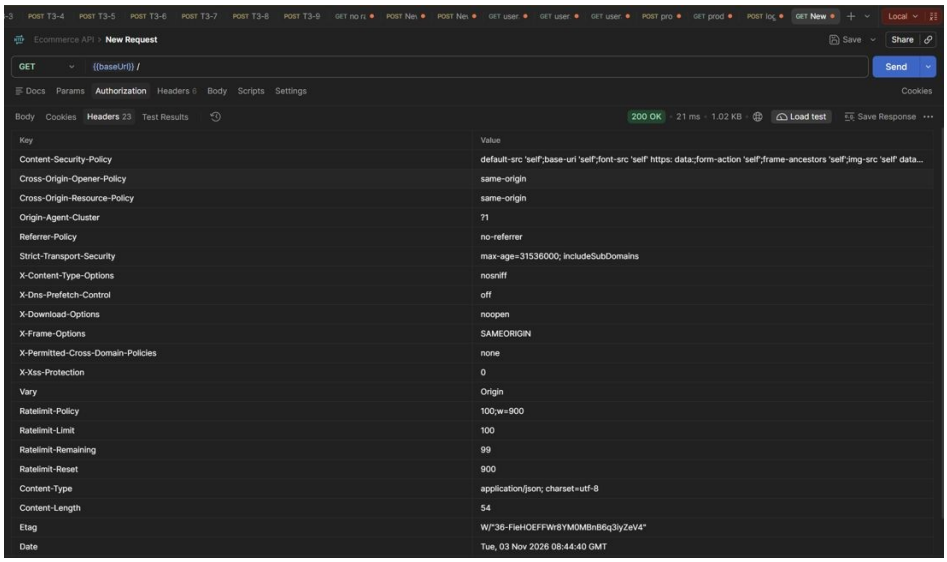
AFTER: allowed origin gets the header (43-cors-allowed-origin-header.png)

Finding 6: No Security Headers

Severity	Medium. Exposes the server type and allows clickjacking and MIME sniffing.
Location	src/app.js
How I tested	Postman: send GET / and read the response headers.
Result before the fix	No protection headers, and X-Powered-By: Express was visible.
How it was fixed	Added helmet() before the routes.
Result after the fix	Security headers are present and X-Powered-By is gone.



BEFORE: headers without Helmet (05-headers-before.png)

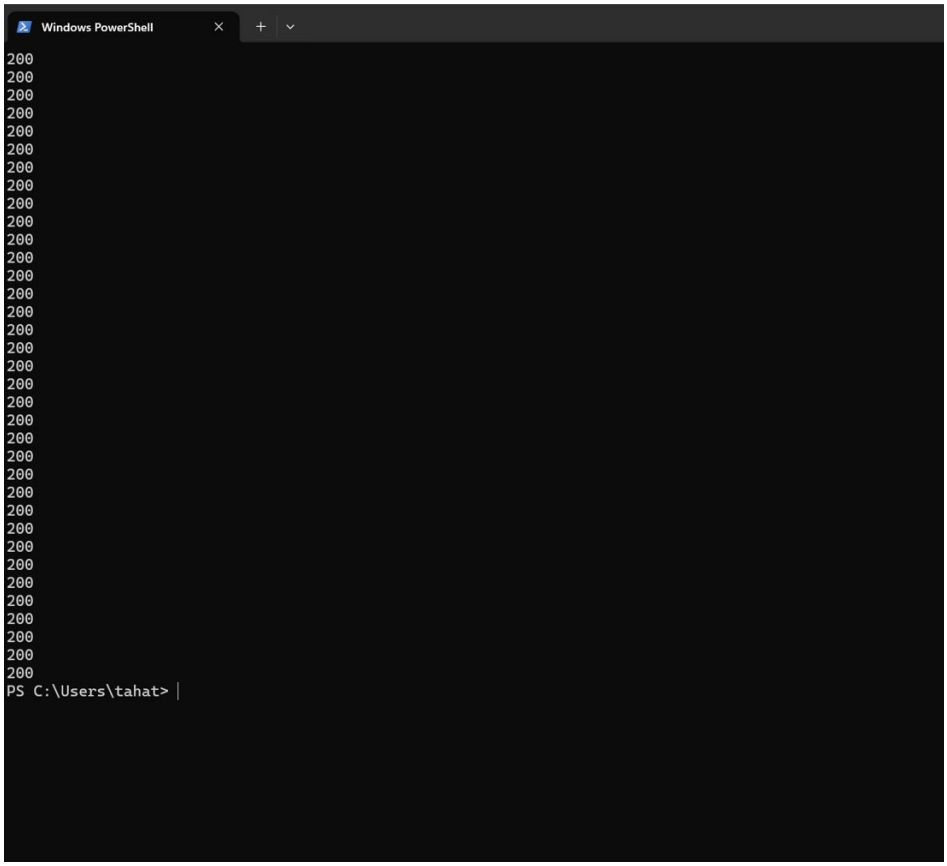


AFTER: Helmet headers present (40-helmet-headers.png)

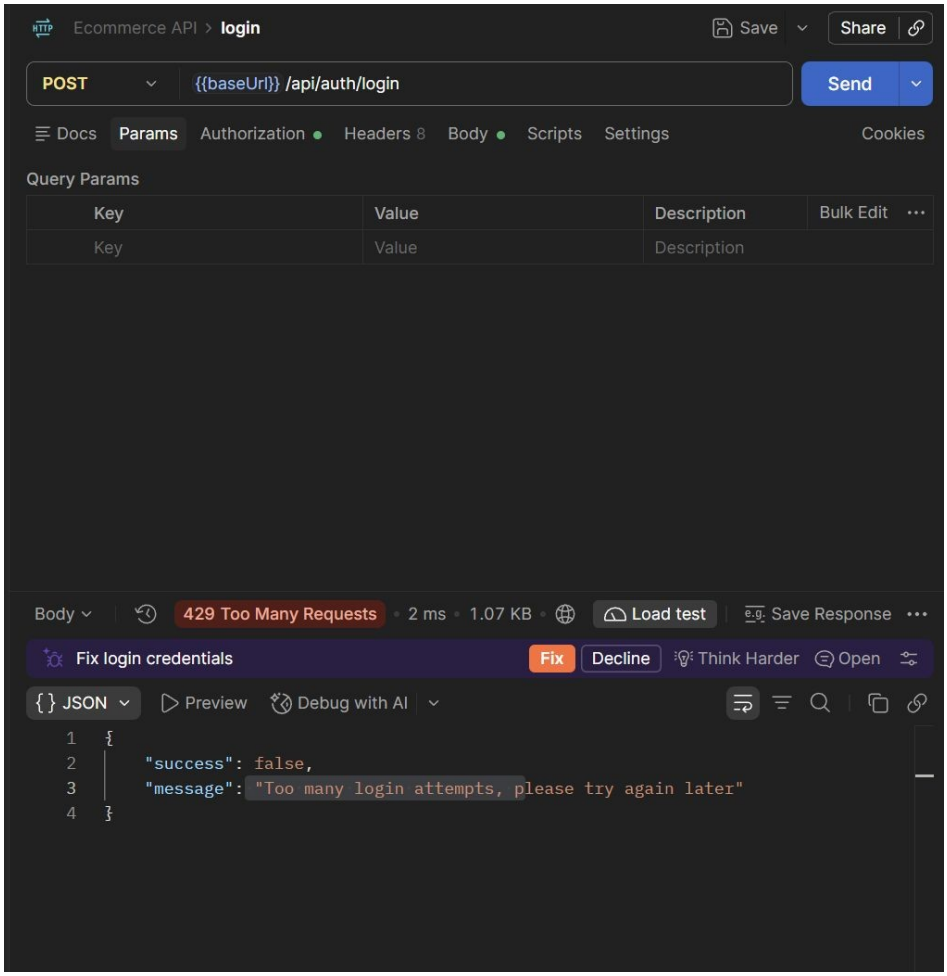
Finding 7: No Rate Limiting

Severity	High. Allows password guessing and abuse.
Location	src/app.js, POST /api/auth/login
How I tested	Postman: send many wrong logins in a row.

Result before the fix	Unlimited attempts were allowed.
How it was fixed	express-rate-limit: a general limit (100 per 15 minutes) and a strict login limit (5 per 15 minutes). Values come from .env.
Result after the fix	The 6th login attempt returns 429 Too Many Requests.



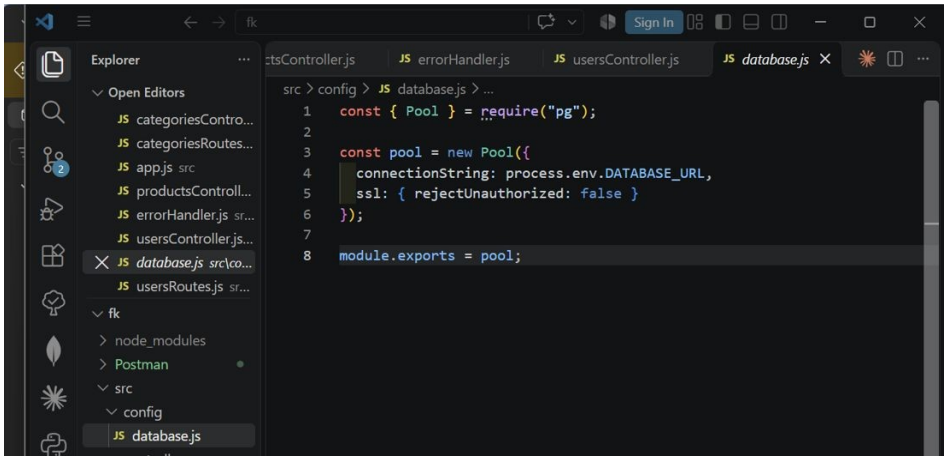
BEFORE: no rate limit (06-no-rate-limit.png)



AFTER: 429 after too many logins (44-login-rate-limit-429.png)

Finding 8: Database TLS Certificate Not Checked

Severity	High. The database connection could be intercepted (man-in-the-middle).
Location	src/config/database.js
How I tested	Code review. After the change, Postman: GET /api/products and POST /api/auth/login.
Result before the fix	The code used ssl: { rejectUnauthorized: false }, so the certificate was not verified.
How it was fixed	Changed it to ssl: { rejectUnauthorized: true } in database.js. Neon uses valid certificates.
Result after the fix	The server still connects to Neon, and GET /api/products and POST /api/auth/login both return 200. The certificate is now checked.



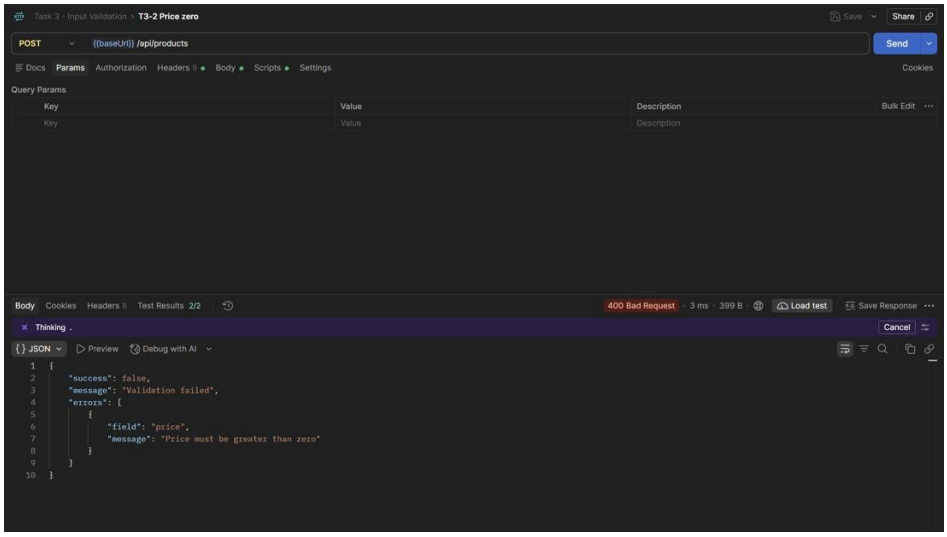
Earlier code evidence of the SSL setting (BEFORE the fix) (08-database-ssl.png)

Finding 9: Unlimited JSON Body Size

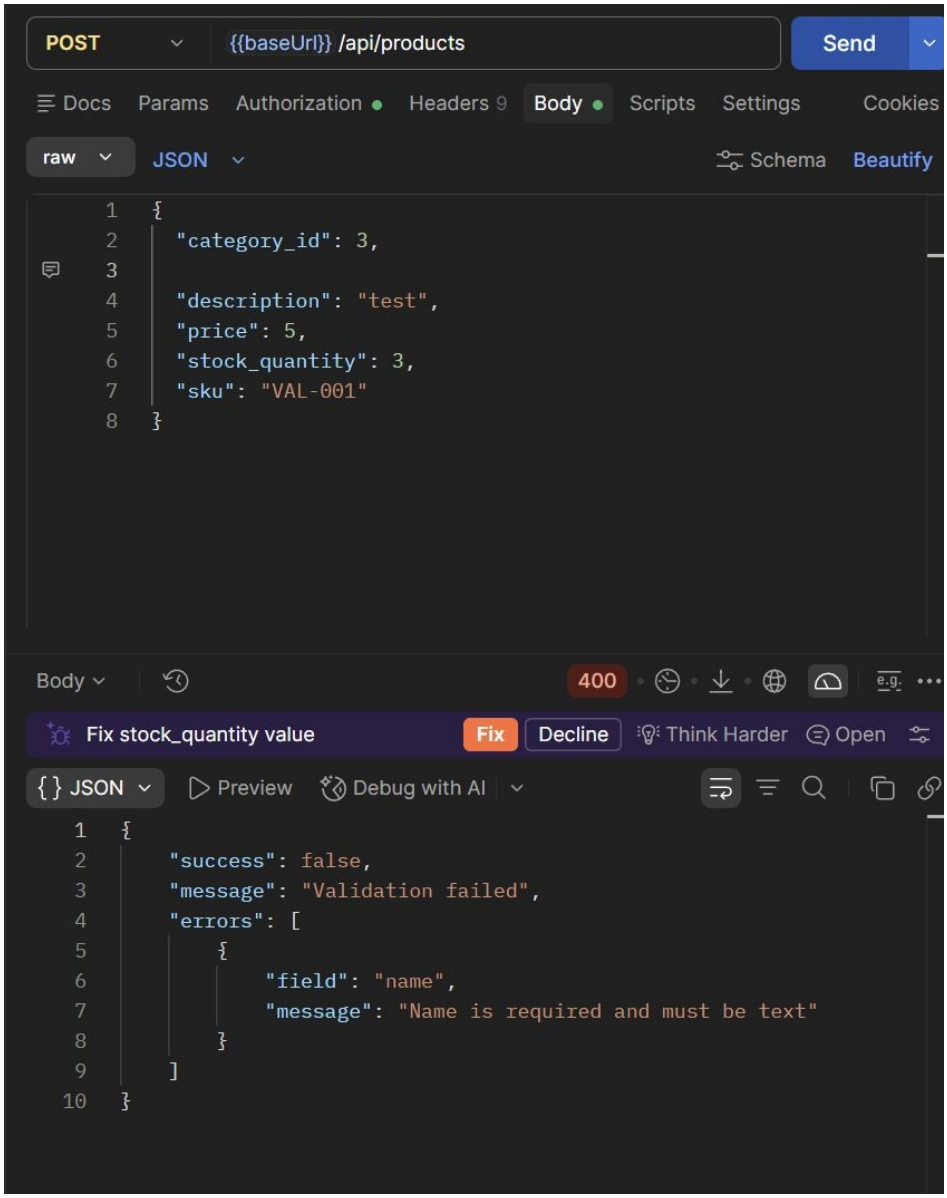
Severity	Low. Very large requests can use up server memory.
Location	src/app.js
How I tested	Code review.
Result before the fix	express.json() had no size limit.
How it was fixed	Set express.json({ limit: "10kb" }).
Result after the fix	Large bodies are rejected with 413 Request body too large.

Finding 10: Weak Input Validation

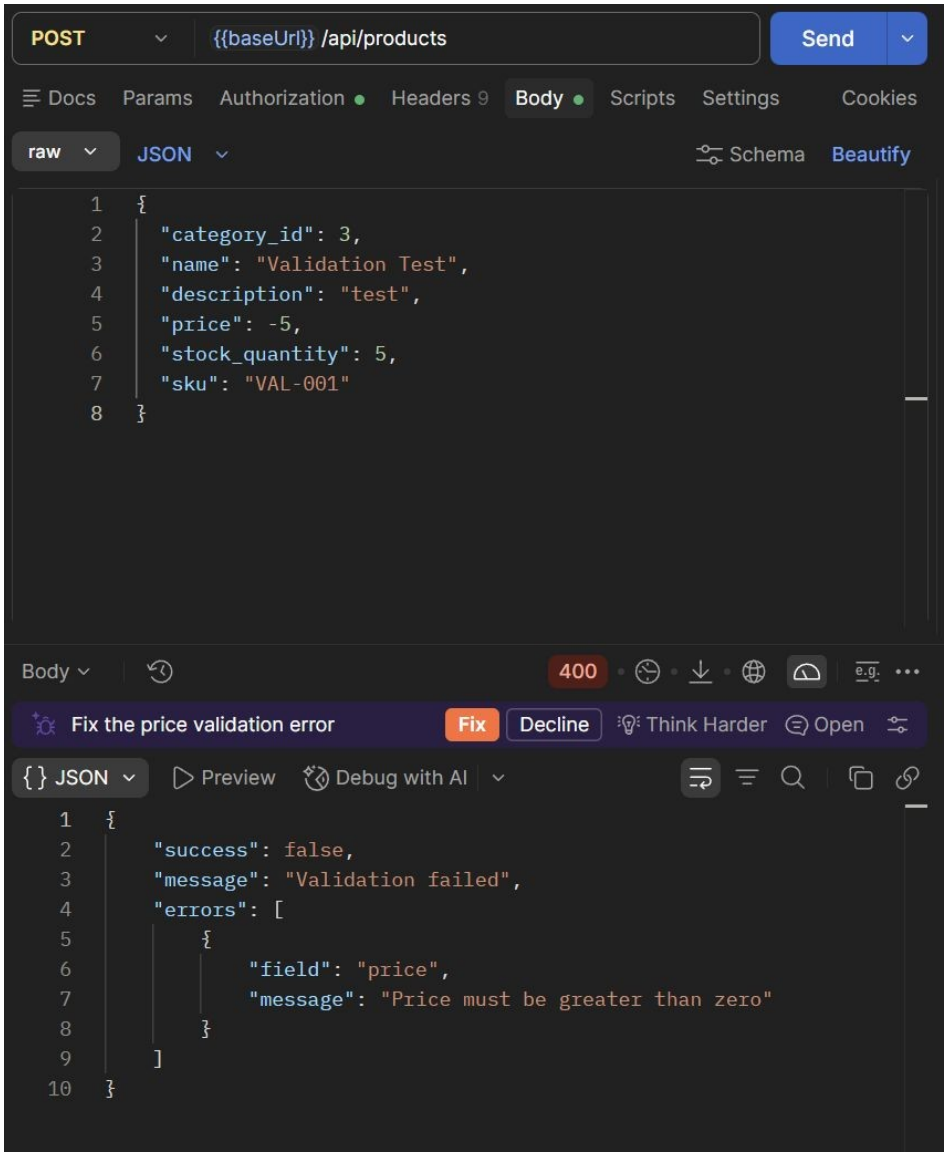
Severity	Medium. Bad data could reach the business logic and the database.
Location	Controllers for products, users and auth
How I tested	Postman: no name, price 0, negative price, negative stock, stock as text, bad email, short password.
Result before the fix	Checks were manual and inconsistent.
How it was fixed	Zod schemas for bodies and for ID parameters. One standard 400 response lists each bad field.
Result after the fix	All invalid inputs return 400 with a clear message and no internal details.



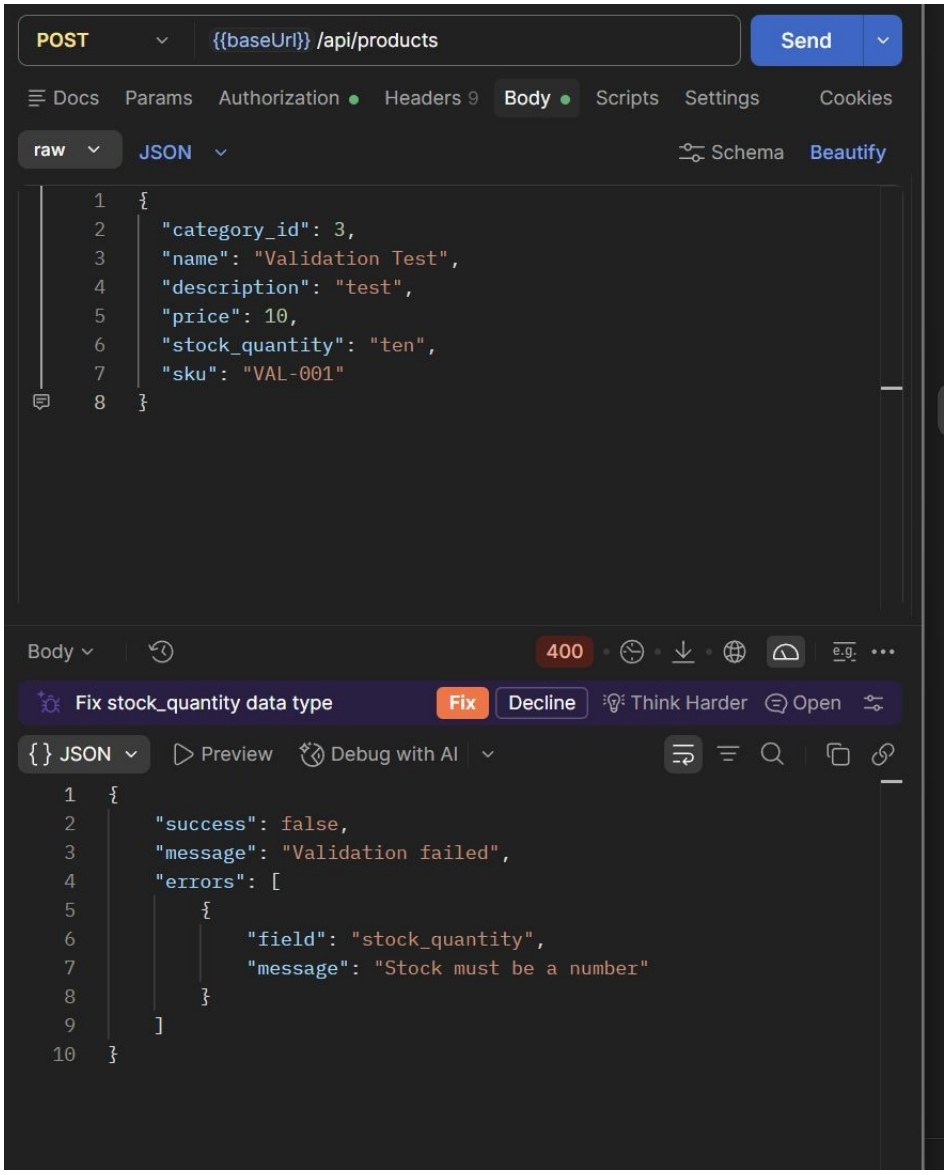
BEFORE: weak price check (11-validation-price.png)



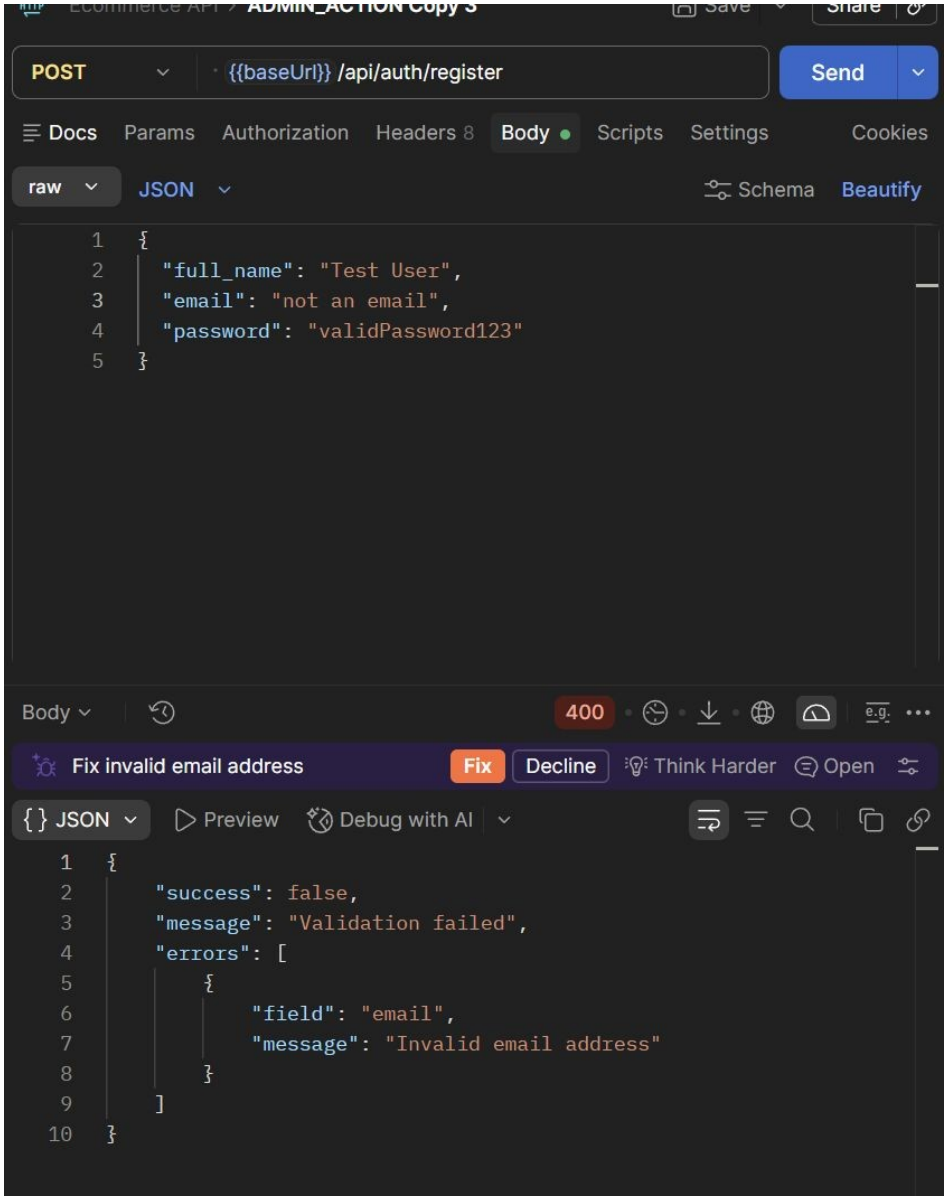
AFTER: product with no name (400) (55-product-no-name-400.png)



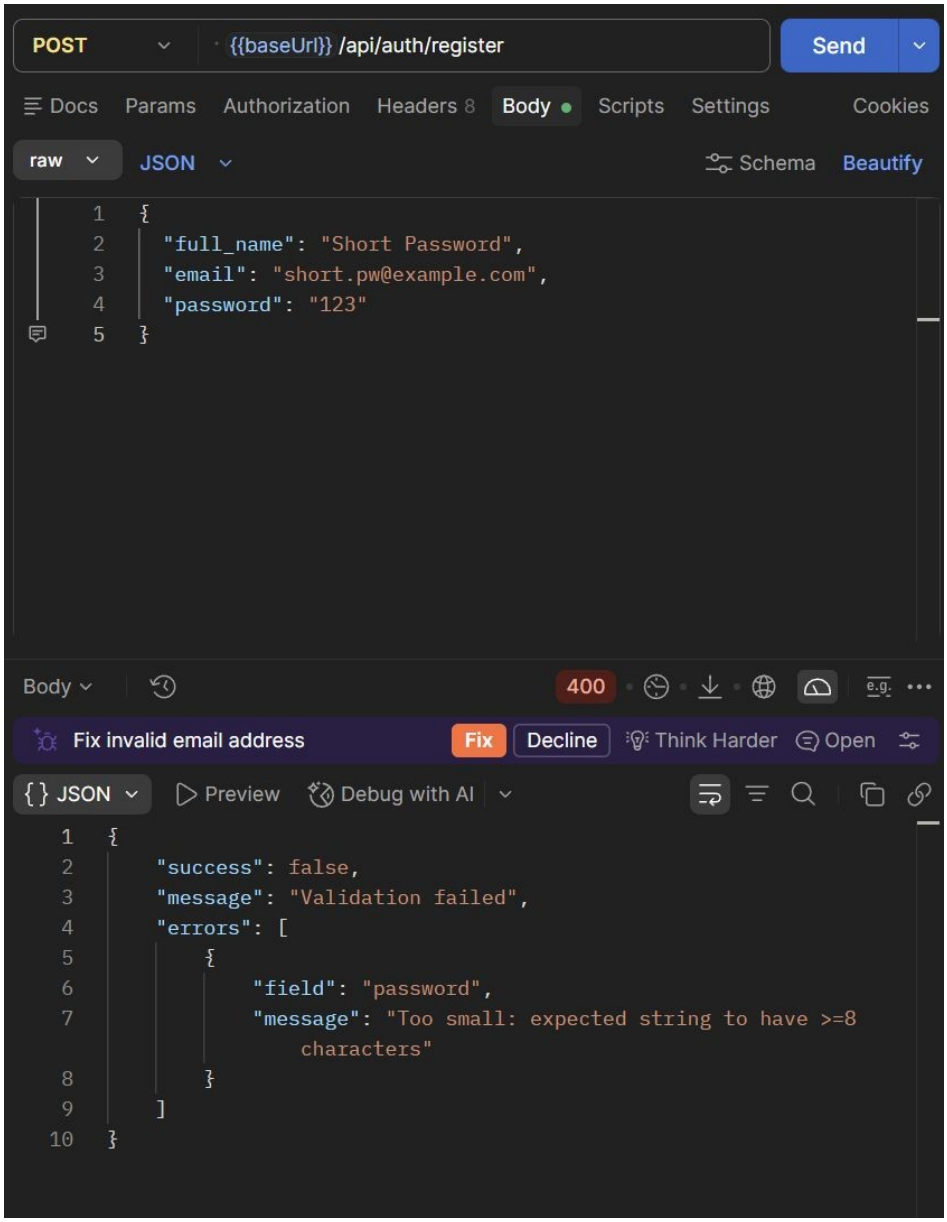
AFTER: negative price (400) (57-product-price-negative-400.png)



AFTER: stock as text (400) (59-product-stock-text-400.png)



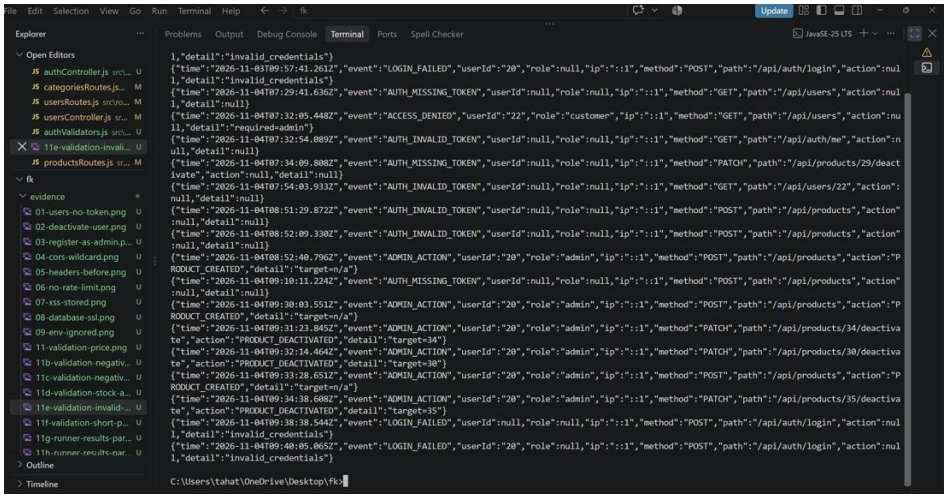
AFTER: bad email (400) (60-register-bad-email-400.png)



AFTER: short password (400) (61-register-short-password-400.png)

Finding 11: No Security Logging

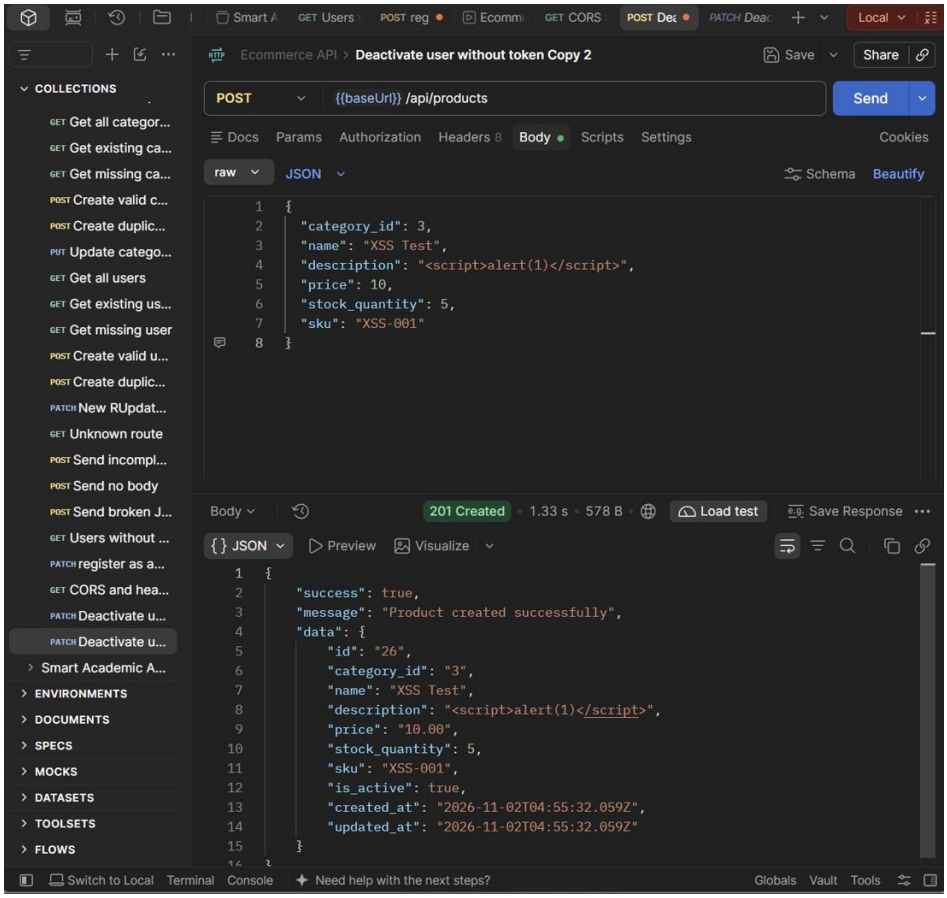
Severity	Medium. Attacks and admin actions left no trace.
Location	Whole project
How I tested	Trigger failed logins, missing tokens, denied access and admin actions, then read logs/security.log.
Result before the fix	Nothing was logged.
How it was fixed	Added utils/logger.js. It logs time, event, user id, role, IP, method and path. It never logs passwords, tokens or hashes.
Result after the fix	Failed logins, 401, 403 and admin actions appear in the log.



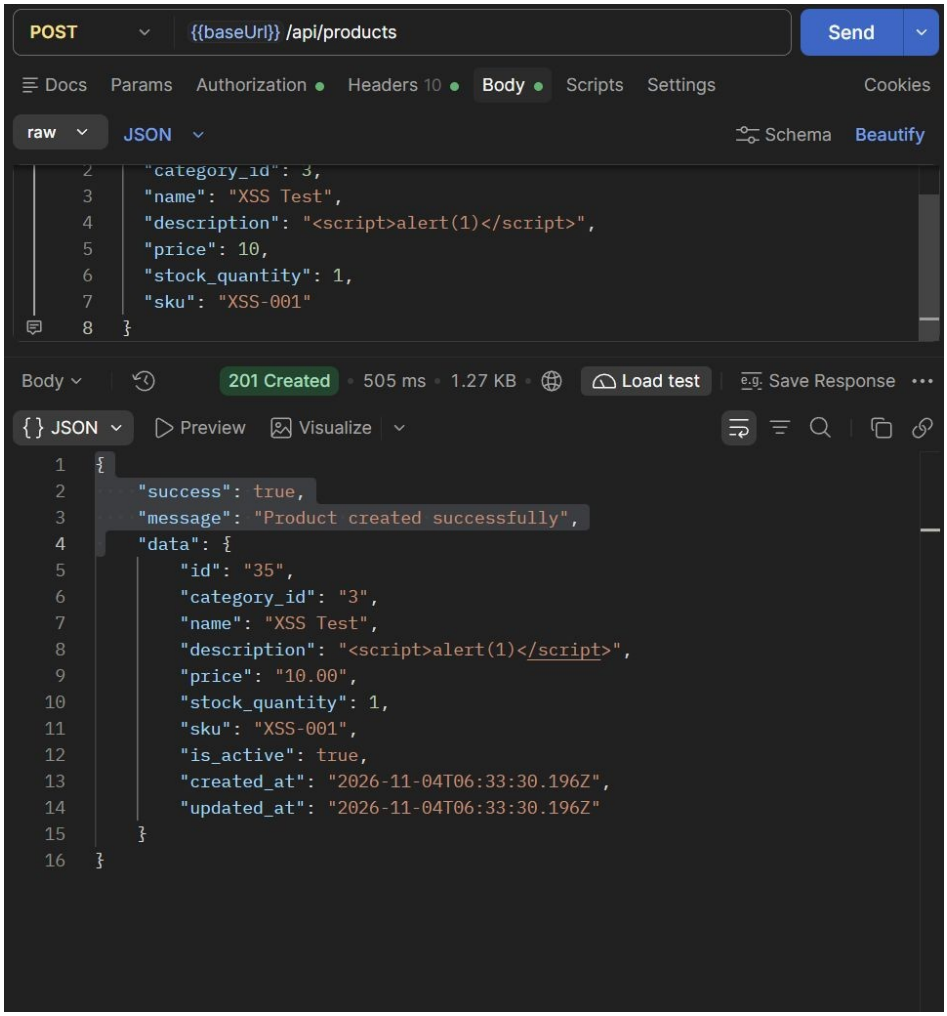
AFTER: security log in the terminal (53-security-log-terminal.png)

Finding 12: HTML Stored in Text Fields (XSS Risk)

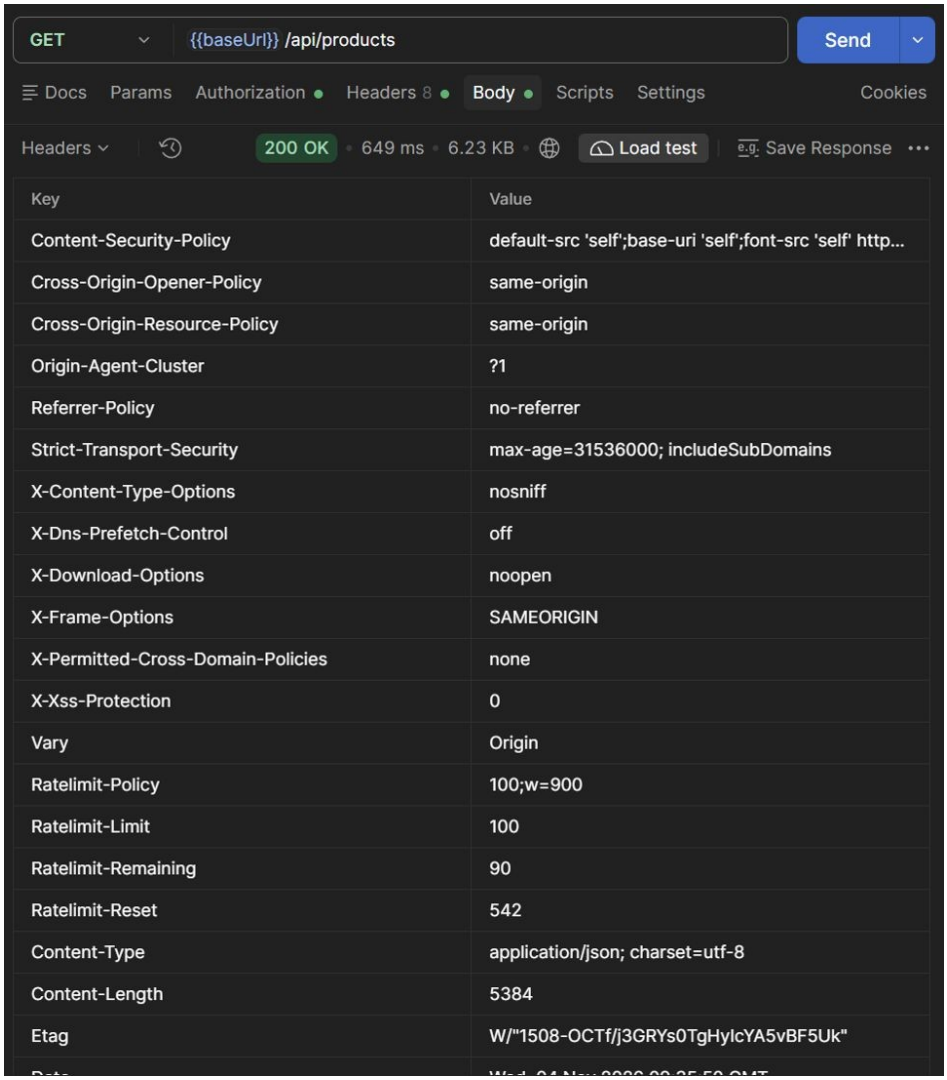
Severity	Low. A front end that renders this as HTML could run an attacker's script.
Location	Product description
How I tested	Postman: send a product description that contains a <script> tag.
Result before the fix	The text was stored with no limits.
How it was fixed	Length limits. Text is stored and returned as plain text in JSON. Helmet adds CSP and nosniff. The front end must show it as text, never as HTML.
Result after the fix	The text is stored as data only, and the response headers include the protection headers.



BEFORE: script text stored (07-xss-stored.png)



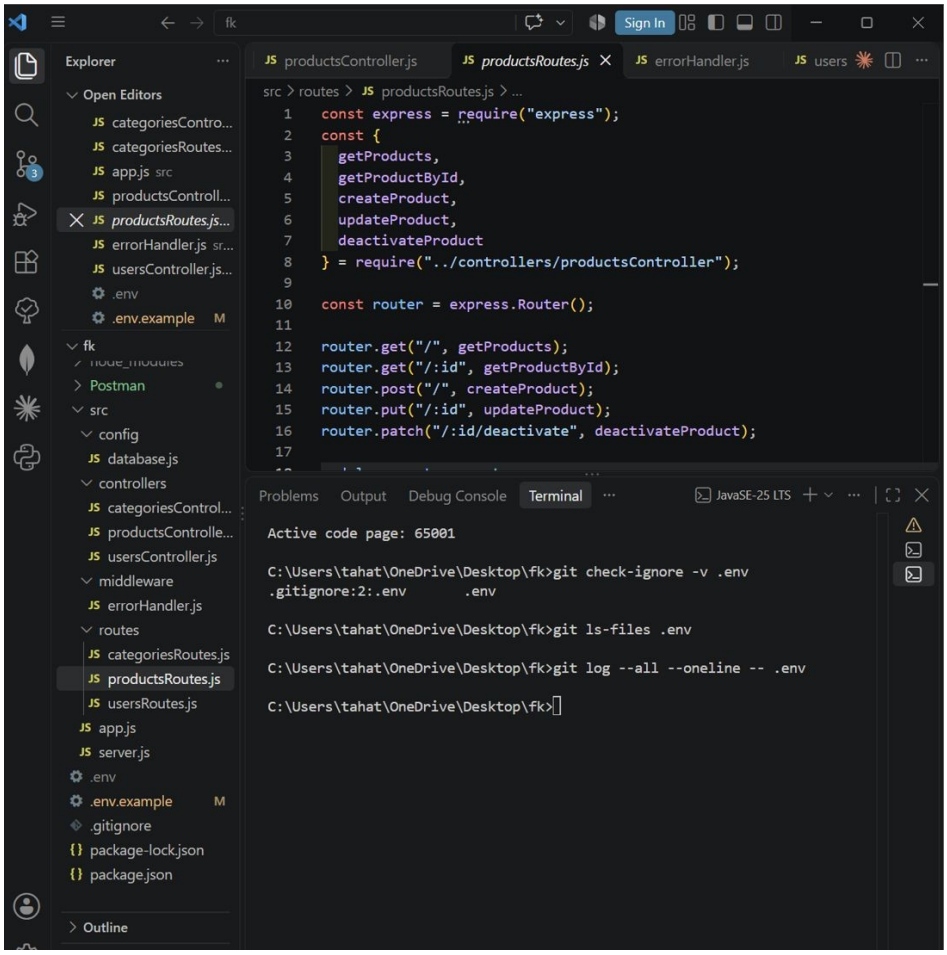
AFTER: stored and returned as plain text (68-xss-stored-as-text.png)



AFTER: protection headers on the response (69-xss-response-headers.png)

Finding 13: .env.example Held a Connection String Template

Severity	Low. It encourages people to put real values in a tracked file.
Location	.env.example
How I tested	File review and git history check.
Result before the fix	The file held a connection string template.
How it was fixed	The file now has variable names with no real values. .env is in .gitignore.
Result after the fix	No real secret is in the repository or in git history.



AFTER: .env is ignored by git (09-env-ignored.png)

Finding 14: Inactive Products Shown in Public List

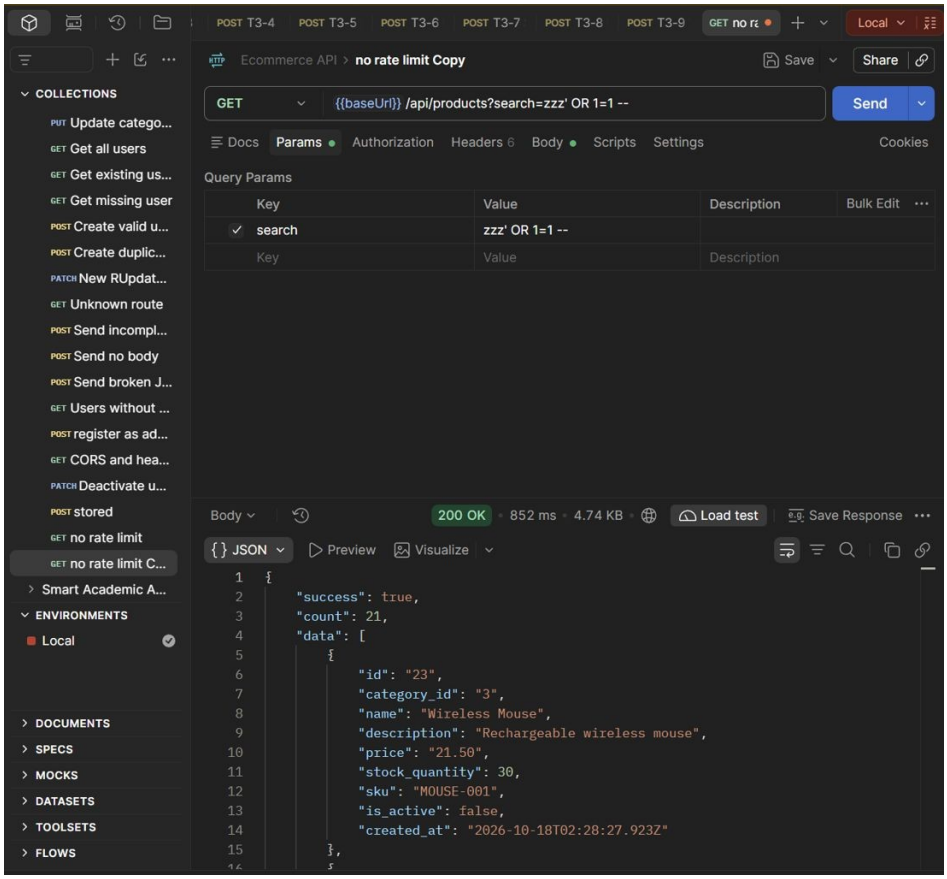
Severity	Low. Shows information about items that are off sale.
Location	GET /api/products
How I tested	Postman: deactivate a product, then call the public list.
Result before the fix	Deactivated products still appeared.
How it was fixed	The list query now uses WHERE is_active = TRUE.
Result after the fix	Deactivated products no longer appear in the public list.

SQL Injection: before and after

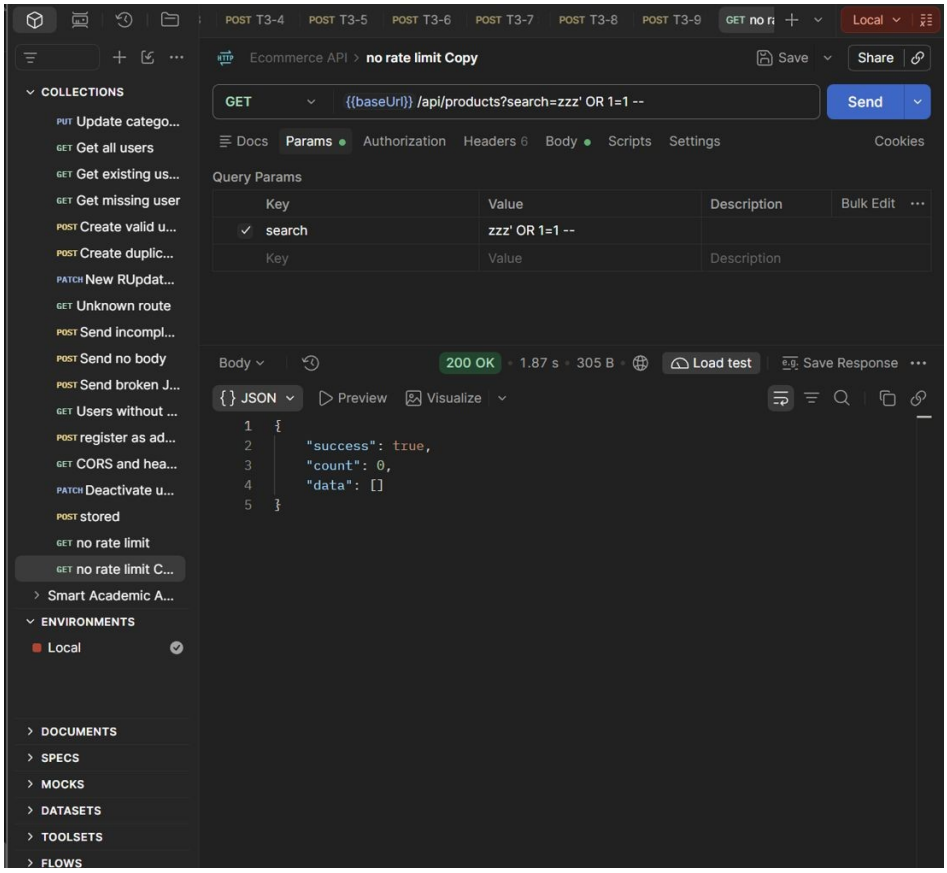
The task asks for one example before and after. The first version already used parameters for values, so I show the ID check that I added. Text such as "1 OR 1=1" is now rejected with 400 before it reaches the database.

Unsafe way	const query = `SELECT * FROM users WHERE email = '${email}'`;
Safe way (used in this project)	pool.query("SELECT * FROM users WHERE email = $1", [email]);

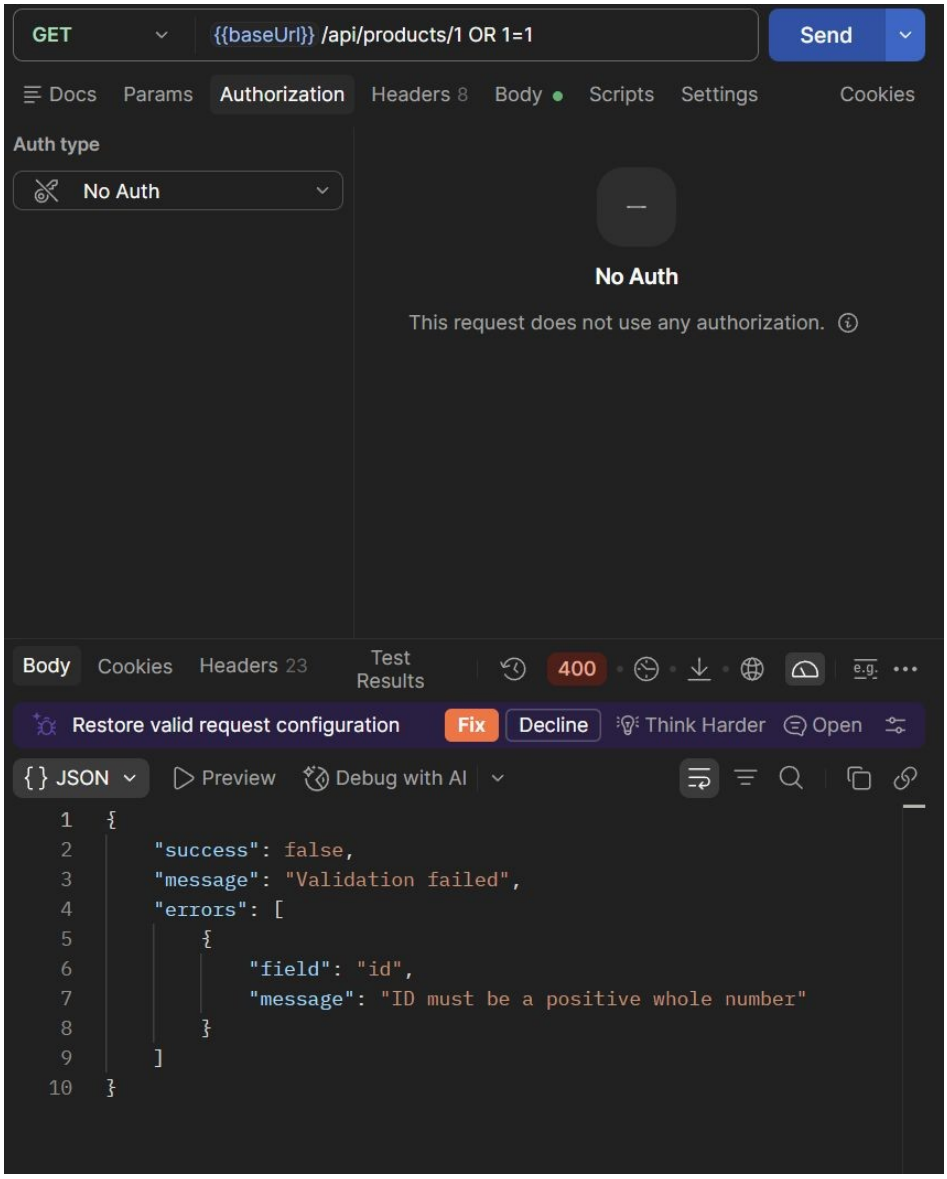
ID check	The ID must match /^[1-9]\d{0,9}$/, otherwise the API returns 400.



BEFORE: SQL text sent in a search field (14-sqli-before.png)



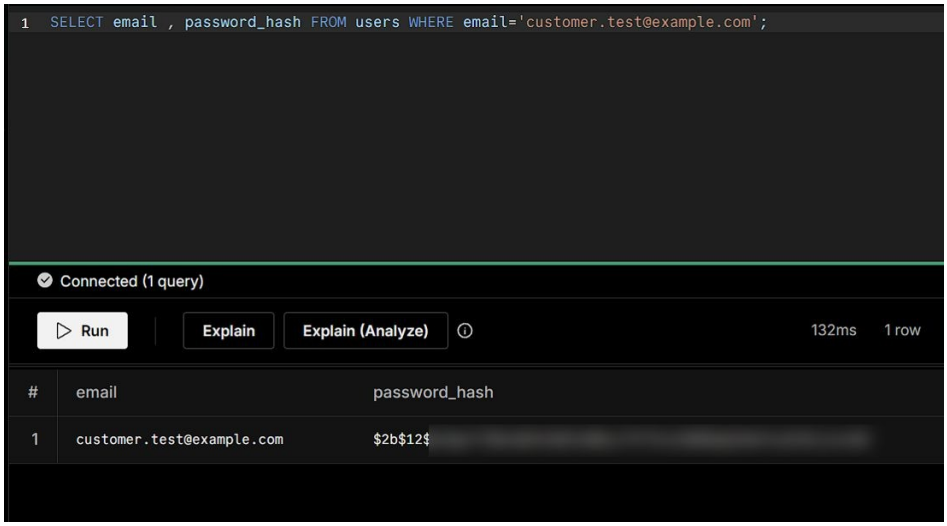
AFTER: the query does not change (15-sqli-after.png)



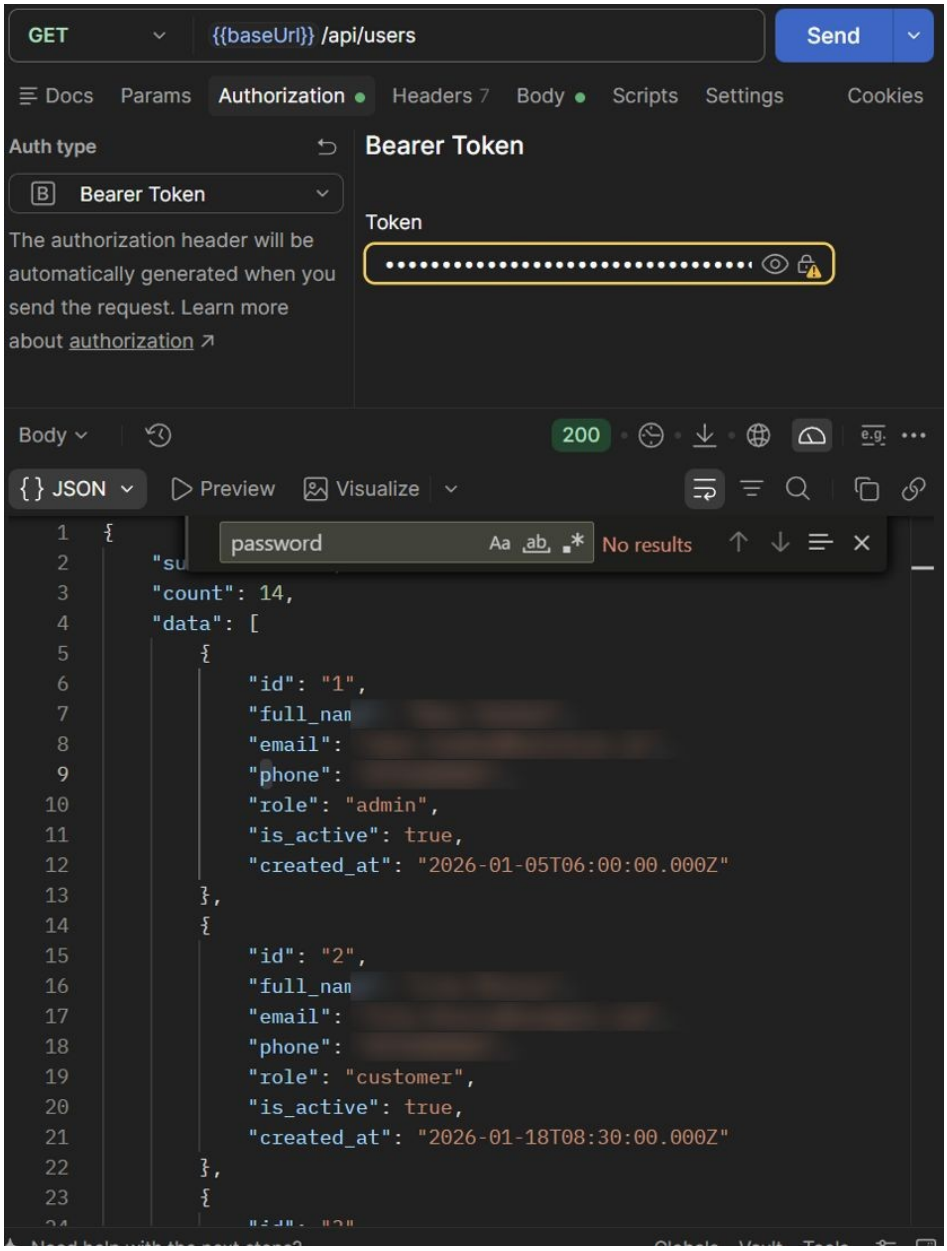
AFTER: SQL text in an ID returns 400 (62-sqli-id-400.png)

Password protection

Passwords are hashed with bcrypt (cost 12) before saving. Login uses bcrypt.compare. A fake hash is compared when the email does not exist, so the API does not show which emails are registered. The password and hash are never returned or logged.



The database holds only a hash, never the real password (64-db-password-hashed.png)



No password or hash in the API response (65-no-password-in-users-response.png)

Three Helmet headers and what they do

Header	Benefit
Content-Security-Policy	Limits where scripts and content can load from. Reduces XSS.
X-Content-Type-Options: nosniff	Stops the browser from guessing file types (MIME sniffing).
Strict-Transport-Security	Tells the browser to use HTTPS only.
X-Powered-By (removed)	Hides that the server runs Express.

4. Mandatory Postman Tests

#	Test	Expected	Screenshot
1	Register a new user	201	B_authentication/20-register-ok-201.png
2	Register a duplicate email	409	B_authentication/21-register-duplicate-email-409.png
3	Login with correct data	200	B_authentication/23-login-ok-200.png
4	Login with wrong password	401	B_authentication/24-login-wrong-password-401.png
5	Protected route without token	401	B_authentication/26-me-no-token-401.png
6	Protected route with invalid token	401	B_authentication/28-me-garbled-token-401.png
7	Customer adds a product	403	C_authorization_and_ownership/36-product-create-customer-403.png
8	Admin adds a product	201	C_authorization_and_ownership/35-product-create-admin-201.png
9	Product with negative price	400	E_validation_and_sql_injection/57-product-price-negative-400.png
10	Bad email or broken JSON	400	E_validation_and_sql_injection/60-register-bad-email-400.png, D_.../67-error-broken-json-400.png
11	Another user's data or order	403 / 404	C_.../46-user-other-record-403.png, C_.../50-order-other-user-404.png
12	Too many login attempts	429	D_.../44-login-rate-limit-429.png
13	Unknown endpoint	404	D_.../41-not-found-404.png
14	password_hash not in results	Field absent	F_passwords_and_xss/65-no-password-in-users-response.png
15	Helmet headers present	Headers present	D_.../40-helmet-headers.png

All 15 tests returned the expected result.

5. Open Items

Tokens last one hour and there is no refresh or revoke list. This is acceptable for this project.

The rate-limit counter is kept in memory, so it resets when the server restarts. A real production system should use a shared store.

6. Conclusion

The API now has login with JWT, role checks, ownership checks, validation, safe SQL, hashed passwords, Helmet, a CORS allow-list, rate limiting, safe errors and security logs. The protections work for both allowed and blocked requests, and the evidence is in the screenshots above.